
RL project: Comparison of DQN and PPO in Snake Game

Angela Bortolato, ID 2156562, Physics of Data

Abstract

This paper evaluates Deep Q-Networks (DQN) and Proximal Policy Optimization (PPO) in a discrete grid-world Snake Game environment. We analyze convergence velocity, sample efficiency, and safety profiles against rule-based baselines under complete and partial information states (POMDP). Results show that while PPO yields superior sample efficiency in fully observable settings, partial observability introduces significant policy instability and aliasing.

1. The Environment Framework

The environment consists of an $N \times N$ grid ($N = 8$ was used) containing static boundaries (walls), a dynamically growing snake body, and a single target (fruit). The game engine steps synchronously based on a discrete action space $\mathcal{A} = \{\text{UP}, \text{RIGHT}, \text{DOWN}, \text{LEFT}\}$.

1.1. Reward Formulation

The scalar reward function $R(s, a)$ is explicitly engineered to balance target exploitation with hard collision avoidance parameters:

$$R(s, a) = \begin{cases} +1.0, & \text{Grid Completion ("Win")} \\ +0.5, & \text{Fruit Consumption} \\ -0.1, & \text{Wall Collision} \\ -0.2, & \text{Self-Biting Collision} \\ 0.0, & \text{Standard Survival Step} \end{cases} \quad (1)$$

1.2. State Transition Exceptions

The environment features three non-standard mechanisms that fundamentally alter policy optimization within an infinite-horizon setup:

- **Tail Truncation:** Self-collisions do not terminate the episode. If the head intersects the ordered body sequence $\mathcal{B} = (b_1, \dots, b_L)$ at segment b_k , all trailing segments from index k to L are deleted. These cells revert to empty and the episode continues with a reduced body length.

- **Perimeter Freezing:** Wall collisions do not trigger a board reset. The head position is frozen, the action is forced to None, and the penalty is repeatedly inflicted for every sequential frame the agent attempts to drive into the boundary.
- **Stochastic Win-Reset:** Completing the board triggers the win reward. To maintain continuous parallel steps, the engine executes an isolated board reset via `get_board()` exclusively for that vector without resetting sister environments.

1.3. The Partially Observable Space (POMDP)

Under full observability, the state input vector represents the total $N \times N \times 4$ one-hot encoded matrix. In contrast, the partially observable case restricts the agent's view to a local square box centered on the snake's head, defined by a mask radius M . For $M = 2$, the agent receives a localized $5 \times 5 \times 4$ observation patch. This configuration introduces spatial state aliasing: completely different physical body configurations yield identical local views, turning the task into a Partially Observable Markov Decision Process (POMDP). The two different configurations are represented in Figure 1

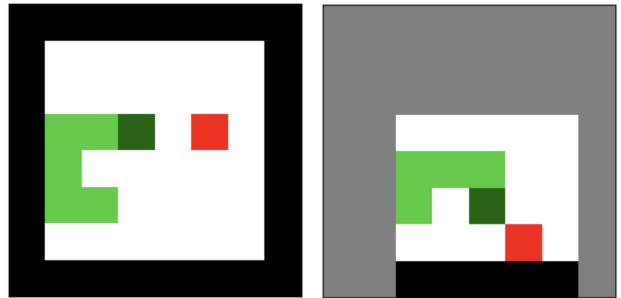


Figure 1. Structural representation of the environment: Full Observability (left) versus Partially Observable 5×5 Masked Configuration (right).

2. Baseline Policies

To benchmark our learning models, three rule-based baseline heuristics were implemented to establish base-

line thresholds across both visibility setups.

2.1. Fully Observable Baselines

Greedy Policy: A heuristic rule that calculates relative target coordinate deltas at each discrete interval. It systematically resolves positional offsets by prioritizing vertical alignment before correcting horizontal path deviations. While highly sample-efficient early on, it is highly vulnerable to local traps and obstacle deadlocks.

ZigZag Policy: A deterministic, hand-crafted routing protocol. The policy uses coordinate modulo arithmetic based on the snake’s head location to guide the agent, completely bypassing wall boundary collisions. It maximizes environmental exploration coverage on grids of even dimension at the structural expense of severe path and step inefficiency.

2.2. Partially Observable Baseline

Semi-Blind Policy: Reads only the local mask tensor. If the fruit channel contains a active target within the 5×5 view, it executes a greedy directional step toward it. If the local window contains no fruit, it falls back to a uniform random step.

3. Reinforcement Learning Methods

3.1. Deep Q-Networks (DQN)

The DQN framework presented in Algorithm 1 optimizes a scalar action-value function $Q(s, a; \theta)$ off-policy under an infinite-horizon setup. To ensure convergence stability and mathematical tracking efficiency, several specific hyperparameter and structural architectural choices were implemented.

3.1.1. NETWORK ARCHITECTURE

Rather than utilizing fully connected layers that flatten spatial features prematurely, a specialized Convolutional Neural Network (CNN) backbone was engineered, as shown in the Figure 2. The input state matches the environment’s representation tensor ($R^{8 \times 8 \times 4}$ under full visibility; $R^{5 \times 5 \times 4}$ under partial visibility). Two sequential convolutional layers with 32 and 64 filters respectively, utilizing 3×3 kernels and **same** padding, extract multi-channel spatial relationships. Crucially, max-pooling layers are omitted to prevent the down-sampling, preserving the exact pixel locations of the head, body, and boundaries. The extracted features are flattened into a 64-unit ReLU dense layer, outputting a linear vector R^4 corresponding to the estimated Q -values for each action in $\mathcal{A}$.



Figure 2. Structural pipeline of the engineered Q-network architecture. Convolutional extraction maps grid features without downsampling, dense layers estimate action values.

3.1.2. VECTORIZED REPLAY BUFFER & MINI-BATCH OPTIMIZATION

To break down temporal correlations in sequential state trajectories, a cyclic **VectorizedReplayBuffer** with a capacity of 100,000 transitions was constructed. At each discrete interval, a synchronous batch of experiences across 128 parallel boards is pushed to the buffer simultaneously. When the buffer size exceeds the mini-batch threshold ($B = 500$), random uniform sampling extracts independent entries. Optimization is conducted using the Huber Loss ($\delta = 1.0$), rather than simple Mean Squared Error (MSE), to clip massive gradient surges resulting from sudden, unexpected wall or tail collisions.

3.1.3. TEMPORAL DEPENDENCIES & EXPLORATION PARAMETERS

- **Exploration Mechanics (ϵ):** Action selection balance relies on an exponential ϵ -greedy schedule ($\epsilon_{\text{start}} = 1.0$, decaying at a steady multiplier per step toward $\epsilon_{\text{end}} = 0.01$). This preserves early-game exploratory paths while enabling high-exploitation behavior late in training.
- **Discount Horizon (γ):** A discount factor of $\gamma = 0.9$ ensures the agent prioritizes immediate food tracking objectives over highly distant, uncertain futures.
- **Soft Target Updates (τ):** To stop the targets from shifting chaotically during optimization updates, a target network $Q(s, a; \theta^-)$ updates its parameters via Polyak averaging: $\theta^- \leftarrow \tau\theta + (1 - \tau)\theta^-$, with $\tau = 0.005$.
- **Optimization Rate:** The model uses the Adam optimizer with a stable learning rate $\alpha = 0.001$ to balance convergence velocity and optimization stability.

3.1.4. LIMITATIONS & POTENTIAL SYSTEM UPGRADES

While standard DQN converges effectively, it preserves explicit algorithmic limitations. First, using the max-operator directly on the target network couples action

Algorithm 1 Parallel Batch DQN

```

1: Initialize networks  $Q_\theta$ ,  $Q_{\theta^-}$ , and Replay Buffer  $\mathcal{D}$ 
2: Initialize environment state array  $\mathbf{s}$  across  $N$  parallel boards
3: for step = 1 to TRAINING_STEPS do
4:   Sample random variable  $r \sim \mathcal{U}(0, 1)$ 
5:   if  $r < \epsilon$  then
6:     Select random actions  $\mathbf{a} \in \mathcal{A}^N$ 
7:   else
8:      $\mathbf{a} \leftarrow \arg \max_a Q_\theta(\mathbf{s})$ 
9:   end if
10:  Execute  $\mathbf{a}$  in environment; collect  $\mathbf{r}$  and  $\mathbf{s}'$ 
11:  Store batch  $(\mathbf{s}, \mathbf{a}, \mathbf{r}, \mathbf{s}')$  into memory buffer  $\mathcal{D}$ 
12:   $\mathbf{s} \leftarrow \mathbf{s}'$ 
13:  if  $|\mathcal{D}| > \text{BATCH\_SIZE}$  then
14:    Sample mini-batch  $(s, a, r, s')$  from  $\mathcal{D}$ 
15:     $y \leftarrow r + \gamma \max_{a'} Q_{\theta^-}(s')$ 
16:    Compute Huber Loss between  $y$  and  $Q_\theta(s, a)$ 
17:    Backpropagate and update weights  $\theta$  via Adam
18:  end if
19:   $\theta^- \leftarrow \tau\theta + (1 - \tau)\theta^-$ 
20:   $\epsilon \leftarrow \max(\epsilon_{\text{end}}, \epsilon \times \epsilon_{\text{decay}})$ 
21: end for
    
```

selection and evaluation, rendering the model vulnerable to overestimation bias. This can be mitigated by upgrading to a **Double DQN** framework, which decouples selection (via the online network) from evaluation (via the target network).

Second, uniform memory sampling treats all trajectories with equal importance, slowing down learning from rare, high-value signals like grid clearances. Implementing **Prioritized Experience Replay (PER)** would focus optimization on high-error states.

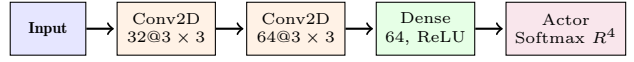
3.2. Proximal Policy Optimization (PPO)

The PPO framework presented in Algorithm 2 optimizes an on-policy Actor-Critic objective function. Unlike off-policy methods that reuse data asynchronously from a memory buffer, PPO relies on a highly synchronized trajectory collection cycle where experiences are gathered, optimized over a fixed number of epochs, and immediately discarded.

3.2.1. NETWORK ARCHITECTURE

The implementation shown in Figure 3 decouples the policy network (Actor, π_θ) from the baseline evaluation network (Critic, V_ϕ) using two completely independent Convolutional Neural Network (CNN) backbones. Both structures mirror the spatial extraction design used in the DQN configuration, feeding the state tensor

Actor



Critic

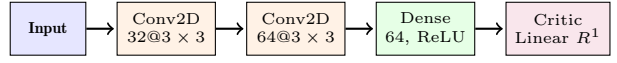


Figure 3. Structural pipeline of the decoupled PPO network architecture. The framework maps observations using two entirely independent networks, treating policy generation (Actor) and value estimation (Critic) as isolated pipelines from initialization to output.

input ($R^{8 \times 8 \times 4}$ full; $R^{5 \times 5 \times 4}$ partial) through sequential layers of 32 and 64 filters with 3×3 kernels and **same** padding, omitting max-pooling layers entirely to preserve coordinate spatial fidelity.

The networks diverge at their dense output heads:

- The Actor Head processes spatial features into a 64-unit ReLU dense layer followed by a 4-unit **softmax** distribution. Actions are selected by sampling directly from this categorical probability matrix.
- The Critic Head maps identical structural features to a single linear scalar node, outputting the anticipated state value expectation $V(s)$.

3.2.2. SURROGATE OBJECTIVES & HYPERPARAMETERS

- **Clipped Policy Loss:** To protect the policy from catastrophic updates, the probability ratio $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ is constrained using a localized surrogate clipping threshold ($\epsilon_{\text{clip}} = 0.2$).
- **Generalized Advantage Estimation (GAE):** Variance across temporal rollout trajectories is minimized using GAE ($\lambda = 0.95, \gamma = 0.90$), balancing immediate temporal differences against multi-step target horizons.
- **Exploration Regularization:** A continuous entropy bonus multiplier ($\beta = 0.02$) is appended directly to the actor loss function to penalize premature distribution saturation and maintain active stochastic exploration.
- **Batch Configurations:** Rollouts are gathered across 128 parallel boards over a horizon length $T = 20$, yielding 2,560 transitions per iteration. This block

Algorithm 2 Parallel Batch PPO

```

1: Initialize Actor  $\pi_\theta$ , Critic  $V_\phi$ , Environment  $\mathcal{E}$ 
2: optimization_step  $\leftarrow 0$ 
3: while optimization_step < TRAINING_STEPS do
4:   Clear rollout storage buffers
5:   for  $t = 1$  to ROLLOUT_STEPS do
6:      $a_t \sim \pi_\theta(s_t)$ ,  $v_t \leftarrow V_\phi(s_t)$ 
7:     Step  $\mathcal{E}$  with  $a_t$ ; collect  $r_t, s_{t+1}$ 
8:     Save transition variables to rollout buffers
9:   end for
10:  Compute advantages  $\hat{A}_t$  via GAE and targets  $G_t$ 
11:  for epoch = 1 to PPO_EPOCHS do
12:    for each mini-batch  $\mathcal{B}$  do
13:      ratio  $\leftarrow \frac{\pi_\theta(a|\mathcal{B})}{\pi_{\text{old}}(a|\mathcal{B})}$ 
14:      Update  $\theta$  via clipped surrogate loss with
        entropy bonus
15:      Update  $\phi$  via Value Function Loss objective
16:      optimization_step  $\leftarrow$  optimization_step + 1
17:    end for
18:  end for
19: end while
    
```

is optimized over 4 training epochs using a mini-batch size of 500 and a stable learning rate $\alpha = 0.001$.

3.2.3. LIMITATIONS & POTENTIAL SYSTEM UPGRADES

PPO’s main limitation stems from its strictly independent dual-backbone setup. Because the actor and critic do not share lower-level convolutional features, the critic must map complex spatial layouts entirely from scratch. In high-dimensional spaces, this increases sample requirements and can be resolved by using a Shared Feature Backbone that splits into policy and value heads only at the final layer.

Additionally, standard on-policy PPO is completely structurally blind to long-term operational history. When operating in hidden, aliased regions where multiple physical layouts appear identical, the model lacks the structural capacity to pass information across steps. This can be resolved by integrating a Recurrent Convolutional Network Architecture (such as a LSTM layer) to process temporal state sequences rather than single static frames.

3.3. Algorithmic Adaptations for POMDP

Transitioning the models into the masked environment required structural modifications:

The CNN input frames seamlessly downscaled from

$(8, 8, 4)$ to $(5, 5, 4)$, pulling the exact bounds directly via the environment’s internal observation layer.

4. Experimental Results and Discussion

Empirical evaluations were conducted by training each agent over a fixed horizon of 30,000 sequential optimization updates across 128 parallelized, vectorized environment boards. To gather statistically sound validation checkpoints, periodic evaluation phases were executed asynchronously every ~ 500 updates. These evaluation trials were performed across 1,000 independent, parallel baseline boards for a strict execution window of 100 steps per episode under a frozen deterministic policy ($\arg \max_a \pi(a|s)$). All primary metrics are compiled into the comparative dashboards shown in Figure 4 and Figure 5.

4.1. Fully Observable Environment Analysis

In the complete information setting, both models successfully converged to near-optimal policies, outperforming or matching the hardcoded baselines (Figure 4):

- **Sample Efficiency:** PPO exhibited exceptional sample efficiency, reaching its performance plateau within $\sim 1,500$ optimization updates. Conversely, DQN required $\sim 5,000$ steps to reach parity, demonstrating the speed advantages of policy-gradient optimization over value iteration.
- **Exploitation and Baselines:** Both models converged onto the performance ceiling of the Greedy Baseline, averaging over 20 fruits per episode and matching its discounted return ($G_0 \approx 1.05$). Conversely, they completely eclipsed the ZigZag Baseline; while the deterministic zigzag path guarantees absolute safety, its spatial routing causes extreme step inefficiencies that yield uncompetitive returns.
- **Exploration Dynamics:** Early in training, PPO experienced a sharp exploration spike, peaking at 8 self-bites per episode around update 1,000. Because PPO samples stochastically from its policy distribution, it prioritizes wall avoidance first, frequently folding onto its own expanding tail before refining global path safety. DQN’s ϵ -greedy mechanism produced a more gradual, conservative rise.
- **Strategic Sacrifices:** Both agents ultimately settled at an identical baseline of ~ 3 self-bites. At body lengths exceeding 20 segments on an 8×8 grid, clipping the tail (penalty -0.2) to secure

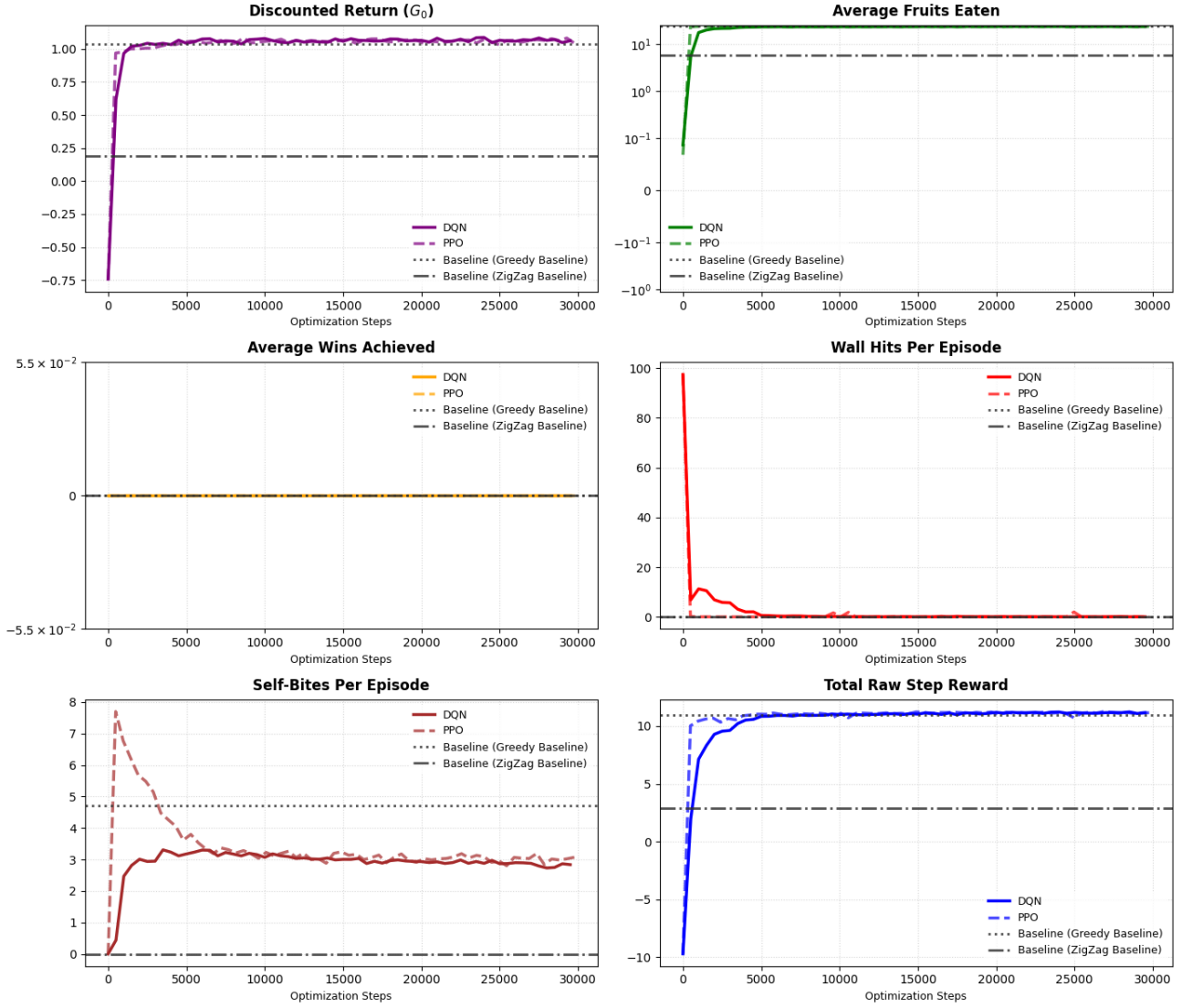


Figure 4. Comparative metric performance dashboards: Fully Observable Environment results.

a fruit (+0.5) is mathematically optimal. Both architectures successfully deduced this calculated risk.

4.2. Partially Observable Environment Analysis

Under the masked local visibility constraint, both algorithms outclassed the rule-based alternative, demonstrating the capacity of deep networks to navigate hidden space configurations (Figure 5):

- Baselines:** Both networks heavily outperformed the Semi-Blind Baseline ($G_0 \approx 0.33$, ~ 16 wall hits). The learned agents utilize structural pattern extraction to maintain efficient traversal strategies
- DQN Dynamics:** DQN displayed rapid early gains but maintained persistent, high-frequency micro-oscillations across its asymptotic plateau. While its off-policy replay buffer smooths out severe gradient shocks, the continuous sampling of aliased, high-penalty transitions during optimization prevents a flat equilibrium.
- PPO Volatility:** PPO exhibited lower-frequency, higher-amplitude volatility, experiencing a sharp performance dip near update 19,000. Because PPO is on-policy, localized entropy reduction can

even when targets fall outside the 5×5 vision mask.

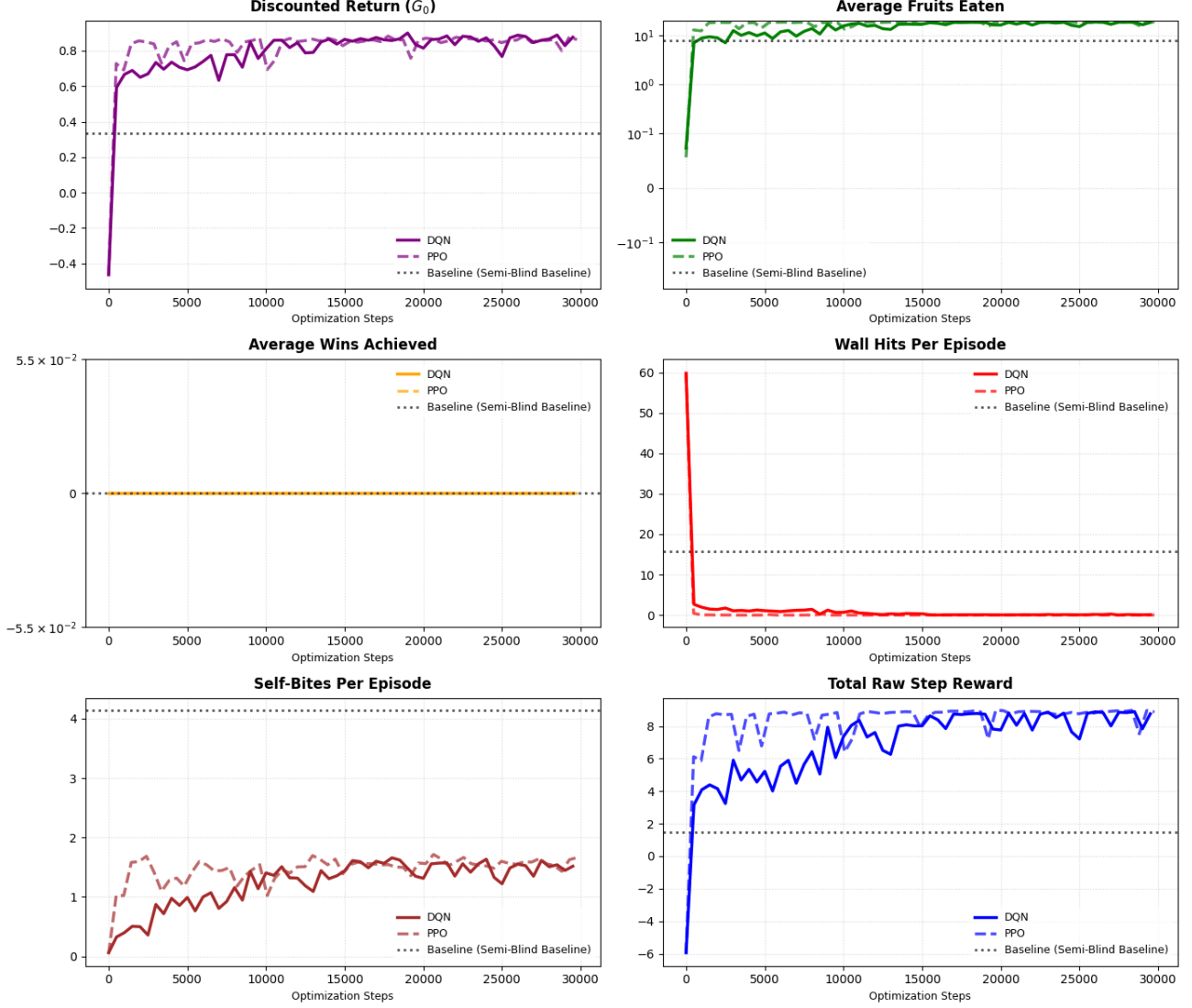


Figure 5. Comparative metric performance dashboards: Partially Observable Environment results.

cause the policy to saturate and become deterministic. When encountering an unseen obstacle, the agent enters sub-optimal loops. Crucially, the entropy bonus ($\beta = 0.02$) preserves an exploratory cushion, allowing the network to break deadlocks and recover performance parity.

- **State Aliasing Instability:** Neither architecture settles into a perfectly flat equilibrium late in training. This shared instability is an inherent property of function approximation under state aliasing. Within a POMDP, distinct board layouts present identical observations inside the local mask. Consequently, gradient updates that correct a maneuver in one layout inadvertently degrade

the policy across corresponding identical-looking intervals, forcing continuous micro-adjustments.

5. Conclusion

This study demonstrates the distinct operational trade-offs between value-based and policy-based methods across different observability settings. While PPO converges rapidly under full visibility, its on-policy nature makes it highly vulnerable to policy collapse under partial observability. DQN, though slower to train, offers much more stable performance when navigating masked, aliased environments.